Lecture 12: Concurrency II: Semaphores, producer-consumer

Professor G. Sandoval

Some slides derived from : Junfeng Yang and Tanenbaum/Bo,

Outline

- Semaphores
- Producer-consumer problem
- Monitors
- Dining Philosophers
- Deadlocks

Semaphore motivation

- Problem with lock: ensures mutual exclusion, but no execution order
- Producer-consumer problem: need to enforce execution order
  - Producer: create resources
  - Consumer: use resources
  - bounded buffer between them
  - Execution order: producer waits if buffer full, consumer waits if buffer empty
  - E.g., $ cat 1.txt | sort | uniq | wc

Semaphore definition

- A synchronization variable that contains an integer value
  - Can't access this integer value directly
  - Must initialize to some value
    - sem_init (sem_t *s, int pshared, unsigned int value)
  - Has two operations to manipulate this integer
    - sem_down (or wait(), P()) (think of lock())
    - sem_up (or signal(), V()) (think of unlock())

Semaphore definition

```
int sem_down(sem_t *s) {
    wait until value of semaphore s
        is greater than 0
    decrement the value of
        semaphore s by 1
}
```

```
int sem_up(sem_t *s) {
    increment the value of
        semaphore s by 1
    if there are 1 or more
        threads waiting, wake 1
}
```

Semaphore uses: mutual exclusion

- Mutual exclusion
  - Semaphore as mutex
  - Binary semaphore: X=1

```
// initialize to X
sem_init(s, 0, X)

sem_down(s);
// critical section
sem_up(s);
```

- Mutual exclusion with more than one resources
  - Counting semaphore: X>1
  - Initialize to be the number of available resources

Semaphore uses: execution order

- Execution order
  - One thread waits for another
  - What should initial value be?

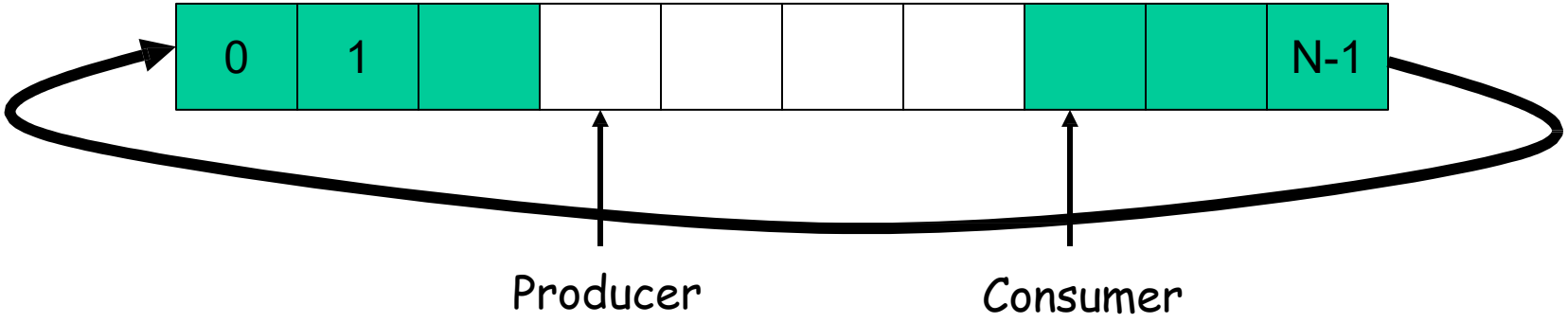
```
//thread 0

... // 1st half of computation
                                  // thread 1
sem_up(s);
                                  sem_down(s);

                                  ... //2nd half of computation
```

Outline

- Semaphores
- Producer-consumer problem
- Monitors
- Dining Philosophers
- Deadlocks

Producer-Consumer (Bounded-Buffer) Problem

- Bounded buffer: size N, Access entry 0… N-1, then "wrap around" to 0 again
- Producer process writes data to buffer
- Consumer process reads data from buffer
- Execution order constraints
  - Producer shouldn't try to produce if buffer is full
  - Consumer shouldn't try to consume if buffer is empty



Solving Producer-Consumer problem

- Two semaphores
  - sem_t full; // # of filled slots
  - sem_t empty; // # of empty slots
- What should initial values be?
- Problem: mutual exclusion?

```
sem_init(&full, 0, X);
sem_init(&empty, 0, Y);
```

```
producer() {
    sem_down(empty);
    … // fill a slot
    sem_up(full);
}
```

```
consumer() {
    sem_down(full);
    … // empty a slot
    sem_up(empty);
}
```

Solving Producer-Consumer problem: final

- Three semaphores
  - sem_t full; // # of filled slots
  - sem_t empty; // # of empty slots
  - sem_t mutex; // mutual exclusion

```
sem_init(&full, 0, 0);
sem_init(&empty, 0, N);
sem_init(&mutex, 0, 1);
```

```
producer() {
    sem_down(empty);
    sem_down(mutex);
    … // fill a slot
    sem_up(mutex);
    sem_up(full);
}
```

```
consumer() {
    sem_down(full);
    sem_down(mutex);
    … // empty a slot
    sem_up(mutex);
    sem_up(empty);
}
```

Outline

- Semaphores
- Producer-consumer problem
- Monitors
- Dining Philosophers
- Deadlocks

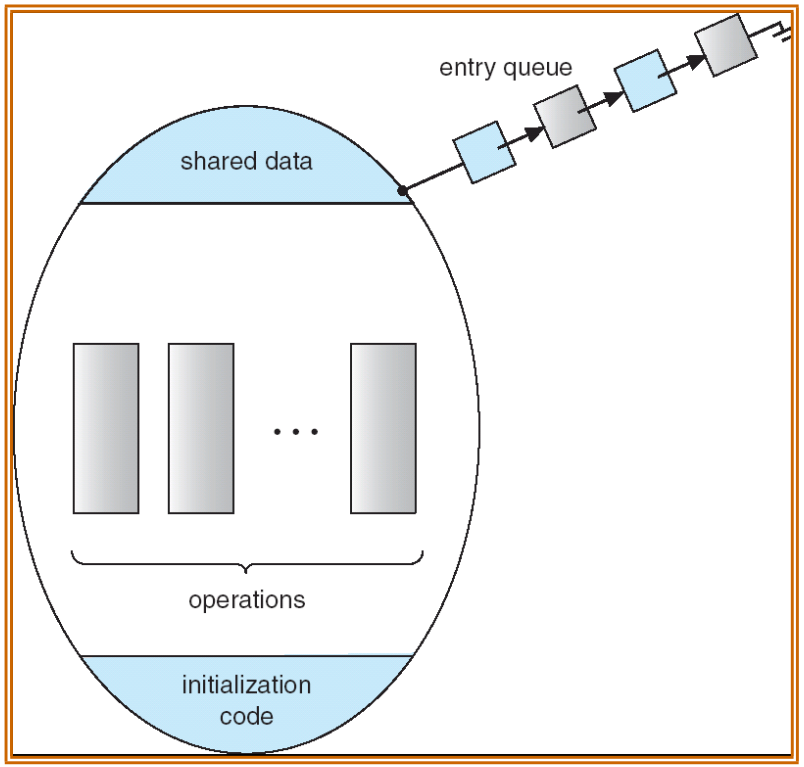
Monitors

- ❑ Background: concurrent programming meets object-
- oriented programming
  - When concurrent programming became a big deal, object-oriented programming too
  - People started to think about ways to make concurrent
  - programming more structured

- ❑ Monitor: object with a set of monitor procedures and only one thread may be active (i.e. running one of the monitor procedures) at a time

Schematic view of a monitor

- Can think of a monitor as one big lock for a set of operations/ methods
- In other words, a language implementation of mutexes



How to implement monitor?

Compiler automatically inserts lock and unlock operations upon entry and exit of monitor procedures

```
class account {
    int balance;
    public synchronized void deposit() {
        ++balance;
    }
    public synchronized void withdraw() {
        --balance;
    }
};
```

```
lock(this.m);
++balance;
unlock(this.m);
```

```
lock(this.m);
--balance;
unlock(this.m);
```

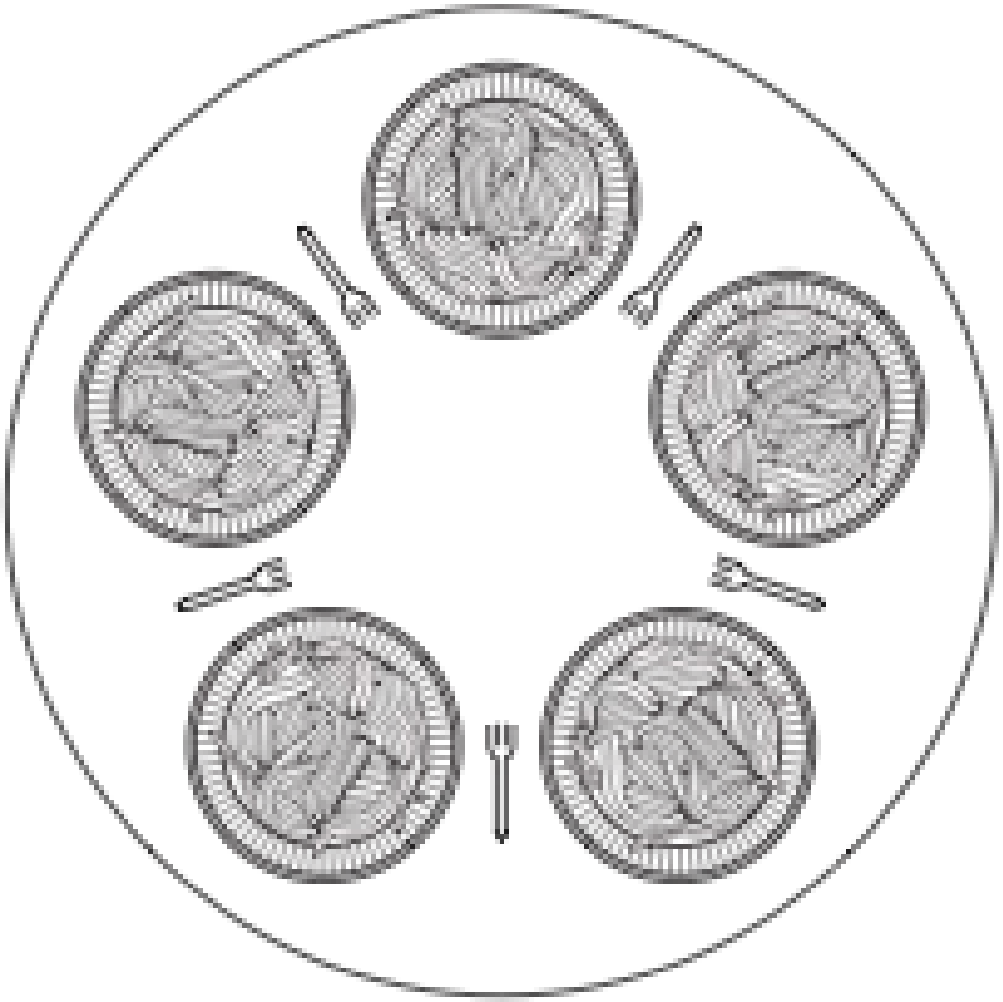
Outline

- Semaphores
- Producer-consumer problem
- Monitors
- Dining Philosophers
- Deadlocks

Dining Philosophers Problem

- Classic synchronization problem that can be solved using many different sychronization primitives
- N philosophers around a table with N forks and N plates are either thinking or eating
- Eating requires two forks, the left fork and the right fork

Delicious Spaghetti



Dining Philosophers Problem

- A naïve solution: when a philosopher gets hungry, just pick up the left fork, then the right fork, waiting until each is available

Dining Philosophers

```
#define N 5                          /* number of philosophers */

void philosopher(int i)              /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                    /* philosopher is thinking */
        take_fork(i);                /* take left fork */
        take_fork((i+1) % N);        /* take right fork; % is modulo operator */
        eat( );                      /* yum-yum, spaghetti */
        put_fork(i);                 /* put left fork back on the table */
        put_fork((i+1) % N);         /* put right fork back on the table */
    }
}
```

Solution to the dining philosophers problem?

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Dining Philosophers Problem

- Some solutions:
  - Have a single mutex. When a philosopher wants to eat, acquire the mutex, then pick up the left and right forks, eat, and release the mutex.
    - But now only one philosopher can eat at a time, which is very inefficient
  - If you can't get your fork, wait a random amount of time before trying again
    - But sometimes we need a deterministic algorithm that will never fail, no matter how many unlikely events happen

Dining Philosophers (1)

```
#define N            5            /* number of philosophers */
#define LEFT         (i+N–1)%N    /* number of i's left neighbor */
#define RIGHT        (i+1)%N      /* number of i's right neighbor */
#define THINKING     0            /* philosopher is thinking */
#define HUNGRY       1            /* philosopher is trying to get forks */
#define EATING       2            /* philosopher is eating */
typedef int semaphore;            /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)           /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                /* repeat forever */
        think( );                 /* philosopher is thinking */
        take_forks(i);            /* acquire two forks or block */
        eat( );                   /* yum-yum, spaghetti */
        put_forks(i);             /* put both forks back on table */
    }
}
```

Figure 2-47. A solution to the dining philosophers problem.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

```
        put_forks(i);                   /* put both forks back on table */
    }
}

void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = HUNGRY;                  /* record fact that philosopher i is hungry */
    test(i);                            /* try to acquire 2 forks */
    up(&mutex);                         /* exit critical region */
    down(&s[i]);                        /* block if forks were not acquired */
}
```

```
}

void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = THINKING;                /* philosopher has finished eating */
    test(LEFT);                         /* see if left neighbor can now eat */
    test(RIGHT);                        /* see if right neighbor can now eat */
    up(&mutex);                         /* exit critical region */
}

void test(i) /* i: philosopher number, from 0 to N–1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Figure 2-47

Tanenbaum

When poll is active, respond at **pollev.com/gsandoval**
Text **GSANDOVAL** to **22333** once to join

What semaphore provides mutual exclusion for the state variable?

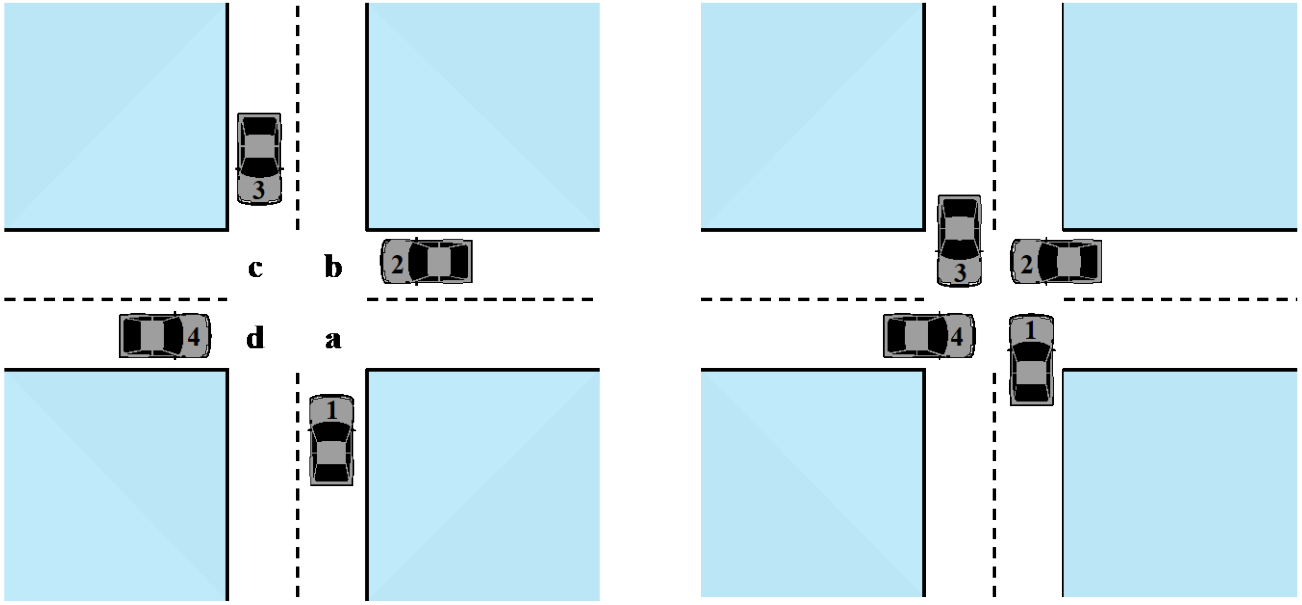
- mutex
- s[n]
- state[n]
- None of the above

Powered by Poll Everywhere
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Outline

- Semaphores
- Producer-consumer problem
- Monitors
- Dining Philosophers
- Deadlocks

deadlock



(a) Deadlock possible

(b) Deadlock

Deadlocks

- Deadlocks are situations in which no work can get done because everyone is waiting to acquire a resource, e.g.:
  - Processes A and B, resources C and D
  - Process A locks C, process B locks D
  - Now A tries to lock D, and B tries to lock C
  - Deadlock – no one can make progress

Deadlocks: Resources

- To be general, we'll just think of abstract *resources* and processes that want to acquire them
- We have two flavors:
  - **Non-preemptible:** once given, they cannot be taken away without cooperation from the process
  - **Preemptible**: we can take it away from the process with no harm done
- For example, RAM is preemptible, because we can just swap the pages to disk without the process noticing

Resource Acquisition (1)

```
typedef int semaphore;
semaphore resource_1;

void process_A(void) {
        down(&resource_1);
        use_resource_1( );
        up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
}
```

(b)

Using a semaphore to protect resources.
(a) One resource. (b) Two resources.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resource Acquisition (2)

```
typedef int semaphore;
    semaphore resource_1;
    semaphore resource_2;

    void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
    }

    void process_B(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
    }
```

(a)

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

(b)

(a) Deadlock-free code.
(b) Code with a potential deadlock.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Deadlock Definition

- We can define a deadlock more formally now:

  A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

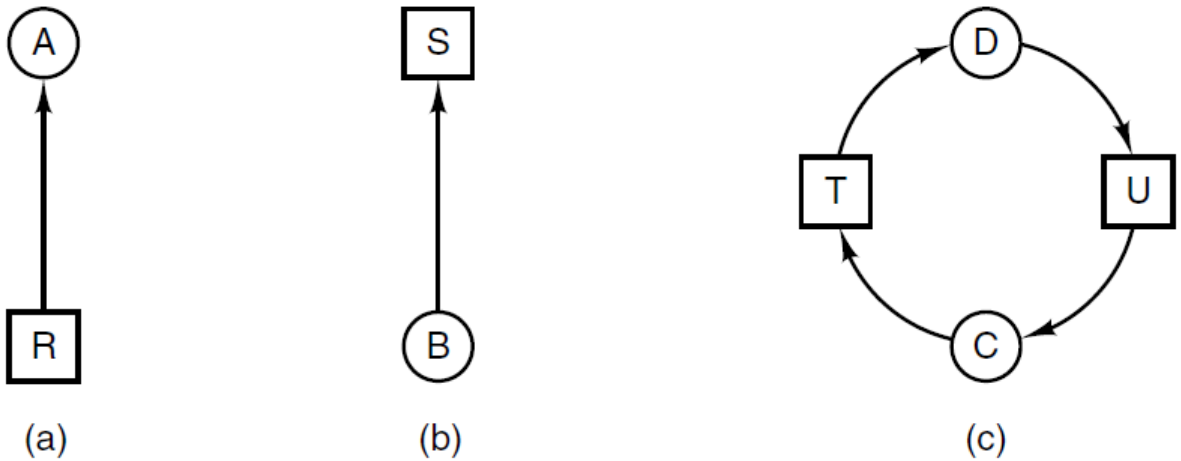
Conditions for Deadlock

Mutual Exclusion	Hold-and-Wait	No Pre-emption	Circular Wait
• only one process may use a resource at a time	• a process may hold allocated resources while awaiting assignment of others	• no resource can be forcibly removed from a process holding it	• a closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain

Modeling Deadlocks

- We can model deadlock conditions as a directed graph
- Two types of nodes: processes and resources
- An edge from process -> resource indicates the process is blocked trying to acquire that resource
- An edge from resource -> process indicates that the resource is currently held by the process

Deadlock Modeling



Resource allocation graphs.

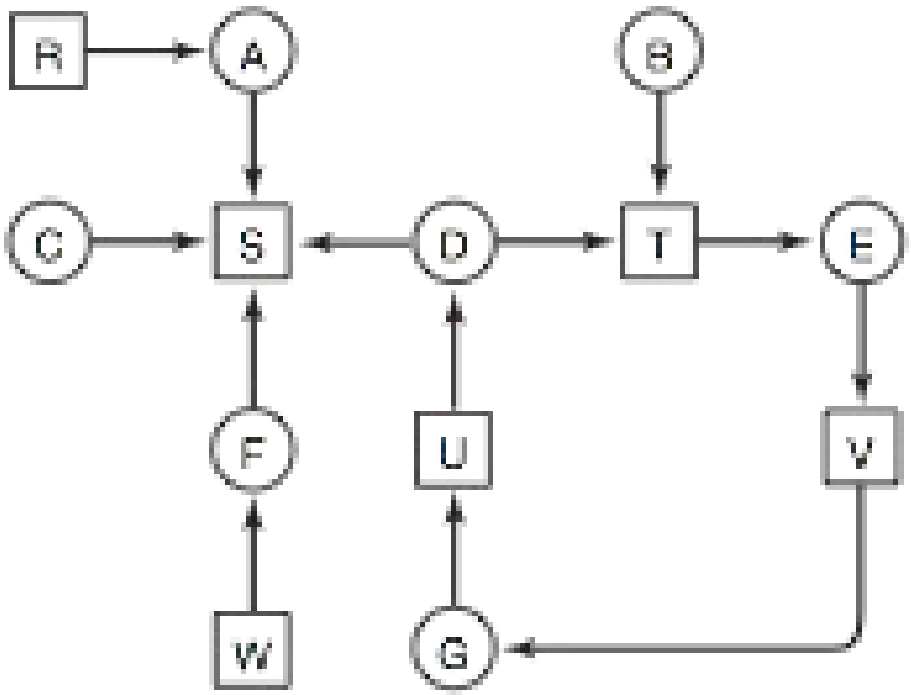
(a) Holding a resource.

(b) Requesting a resource.

(c) Deadlock.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

More Complicated



Starvation

- Processes need resources
- OS allocates resources based on a policy
- Sometimes this policies lead to a process never getting the resource => **Starvation**
- Can be avoided using **FIFO** policy
- Examples:
  - Dining Philosophers
  - Printing a large file

Livelock

```
void process_A(void) {
        acquire_lock(&resource_1);
        while (try_lock(&resource_2) == FAIL) {
                release_lock(&resource_1);
                wait_fixed_time();
                acquire_lock(&resource_1);
        }
        use_both_resources( );
        release_lock(&resource_2);
        release_lock(&resource_1);
}
```

```
void process_B(void) {
        acquire_lock(&resource_2);
        while (try_lock(&resource_1) == FAIL) {
                release_lock(&resource_2);
                wait_fixed_time();
                acquire_lock(&resource_2);
        }
        use_both_resources( );
        release_lock(&resource_1);
        release_lock(&resource_2);
}
```

Summary

- **Deadlock**: Situation in which a set of threads/processes cannot proceed because each requires resources help by another member of the set.
- **Detection and Recovery**: recognize deadlock after it has occurred and break it.
- **Avoidance**: don’t allocate a resource if it would introduce a cycle.
- **Prevention**: Design resource allocation strategies that guarantee that one of the necessary conditions never holds.
- Code Concurrent programs carefully. This only helps prevent deadlock over resources managed by the program, not OS resources.
- Ignore the possibility! (Most Oses use this option)